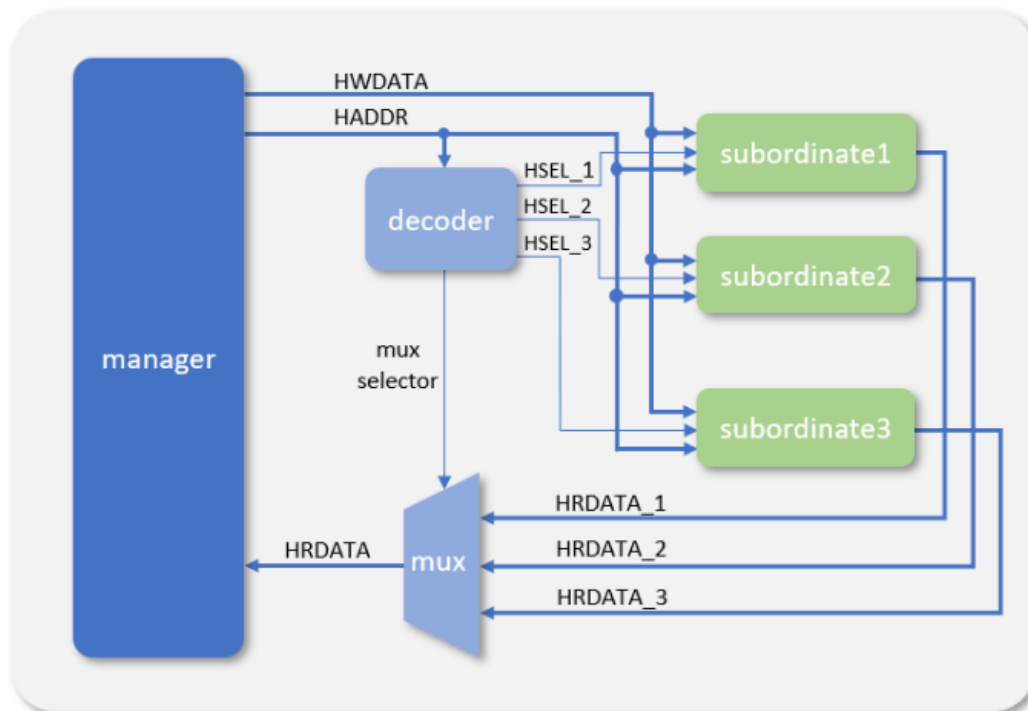


AMBA AHB-Lite Protocol

Design, Implementation & Verification Report



Overview of AHB-Lite

AHB-Lite is a simplified version of the AMBA AHB (Advanced High-performance Bus) protocol, developed by ARM as part of the AMBA (Advanced Microcontroller Bus Architecture) family. It is designed for high-speed, high-bandwidth communication within a system-on-chip (SoC).

Key Features

- Single master support (unlike AHB, which allows multiple masters).
- Supports burst transfers (sequential data transfers).
- Supports pipelined operation with separate address and data phases.
- Simplified interface and lower area and power than full AHB.
- Widely used in embedded systems and microcontrollers.

Uses and Importance

Uses

- On-chip communication between processor cores and memory or peripherals.
- Used in SoC designs, especially for ARM Cortex-M based microcontrollers.
- Interfaces like AHB to APB bridges use AHB-Lite.
- Internal interconnect of memory-mapped peripheral devices.

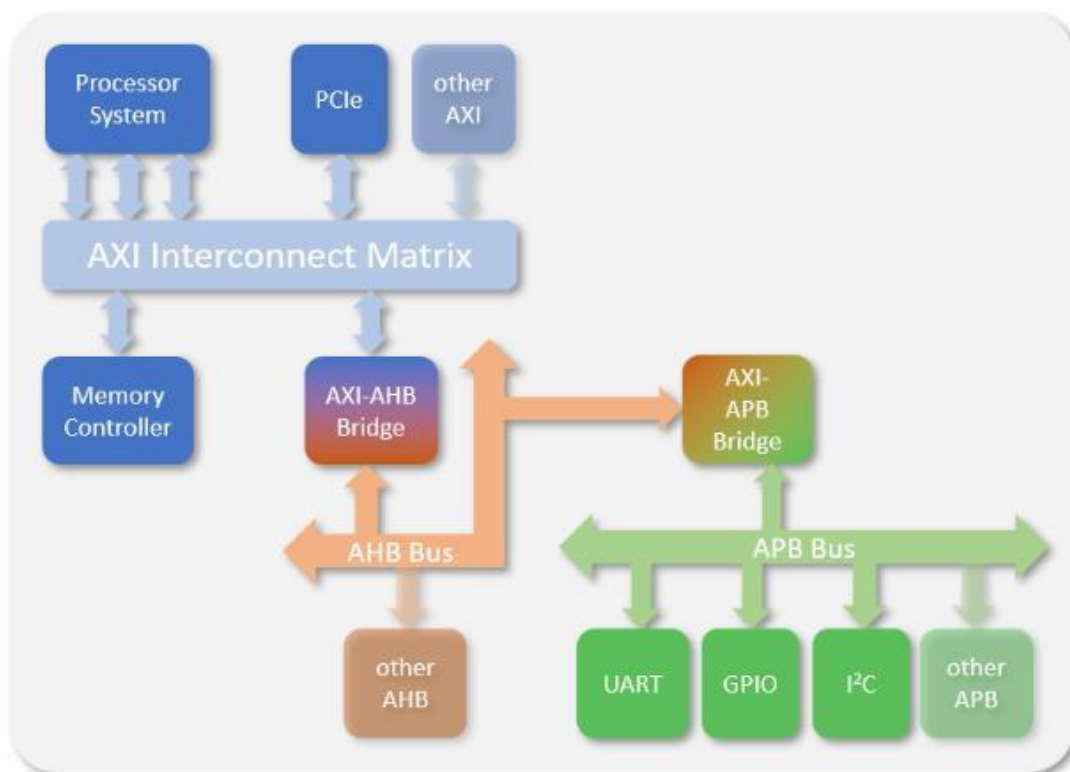
Importance

- Simple yet efficient for single-master systems.
- Pipelining improves performance without the complexity of multi-master arbitration.
- Maintains compatibility with the AMBA ecosystem and IP reuse.
- Offers predictable timing and deterministic performance.

Applications of AHB-Lite

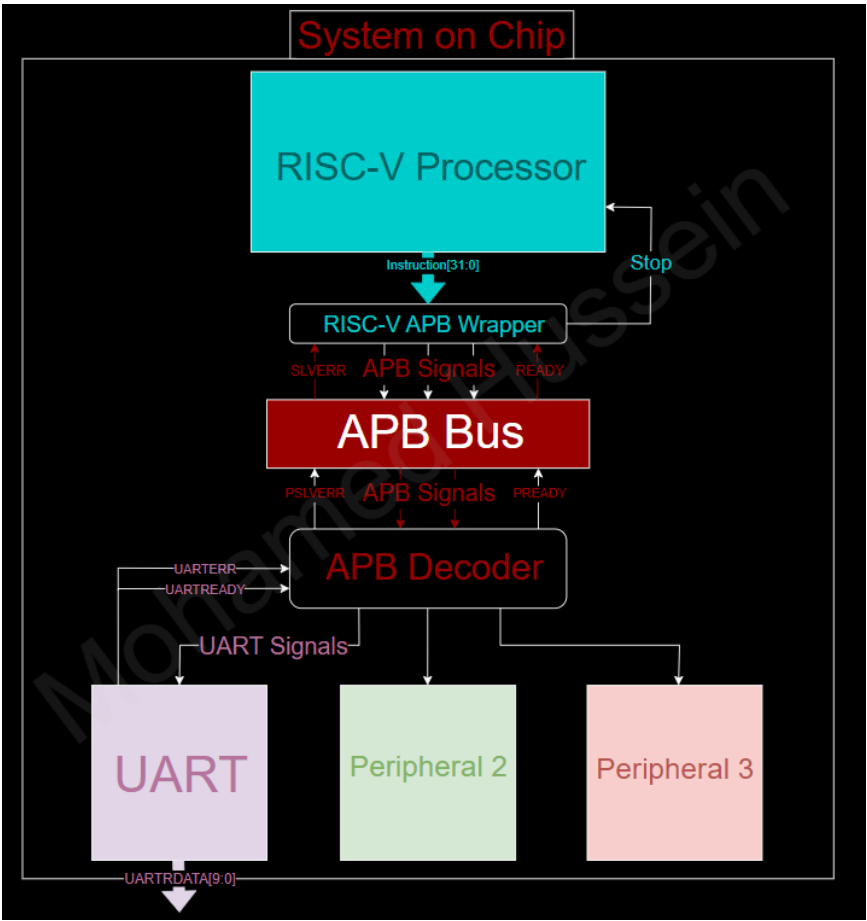
- ARM Cortex-M based microcontrollers (e.g., STM32, NXP LPC).
- Embedded SoCs with single bus master (CPU core).
- Internal memory/peripheral interfacing (RAM, GPIO, UART, etc.).
- Base interconnect for bridging to other buses (e.g., AHB to APB).

How Do We Use It?



The diagram shows the main system bus is AXI, with its interconnect fabric. A bridge connects one of the AXI ports to an AHB bus, though multiple buses might be supported. On the AHB bus, another bridge connects an APB bus, though it might have been connected directly to an AXI interconnect port. In general, though, there would be a hierarchy, with lower bandwidth and latency-tolerant peripherals further away from the main system bus, and high-speed, low-latency peripherals nearer the main bus.

Note: a real-life example is available in a System on Chip repository, where a RISC-V processor is connected to multiple peripherals (UART) using an APB bus.



Understanding AHB-Lite Data Transfer

Address Phase vs. Data Phase

AHB-Lite operates using pipelined bus cycles, meaning address and control signals for the next transfer can be issued while data is still being transferred for the current one. This is enabled by the address phase and data phase.

Phase	Description
Address Phase	In this phase, the master places the address and control signals (HADDR, HTRANS, HSIZE, HBURST, HWRITE) on the bus.
Data Phase	The slave either provides HRDATA (read) or accepts HWDATA (write). This happens in the cycle after the address phase.

Pipelining allows the next address to be issued without waiting for the current data to complete, improving throughput significantly.

Transfer Types

AHB-Lite supports different types of burst transfers to accommodate a wide range of data movement patterns. The type of transfer is determined by the HBURST signal.

HBURST[2:0]	Transfer Type	Description
3'b000	SINGLE	A single transfer of data.
3'b001	INCR (Incrementing)	A burst of unspecified length. Address increases after each beat.
3'b010	WRAP4	4-beat burst with wrapping. Address wraps around on boundary.
3'b011	INCR4	4-beat burst with incrementing addresses.
3'b100	WRAP8	8-beat burst with wrapping.
3'b101	INCR8	8-beat burst with incrementing addresses.
3'b110	WRAP16	16-beat burst with wrapping.
3'b111	INCR16	16-beat burst with incrementing addresses.

SINGLE (HBURST = 3'b000)

- Transfers one data word (8/16/32-bit).
- Common in simple read/write transactions.
- HTRANS = NONSEQ only.

INCR (HBURST = 3'b001)

- Used for undefined-length bursts.
- Address auto-increments by the transfer size each cycle.
- Requires HTRANS = NONSEQ followed by SEQ on next beats.
- Master must signal the end using HTRANS = IDLE or HTRANS = BUSY.

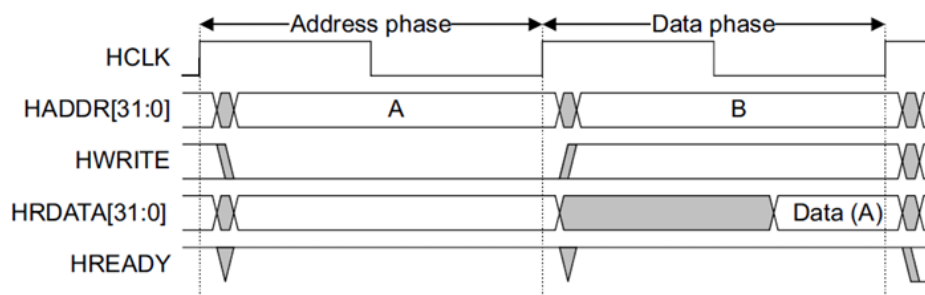
INCR4, INCR8, INCR16 (HBURST = 3'b011/101/111)

- Fixed-length incrementing bursts of 4, 8, or 16 beats.
- Address increments linearly.
- Used when burst length is known in advance.
- Better for optimizing bandwidth in memory systems.

WRAP4, WRAP8, WRAP16 (HBURST = 3'b010/100/110)

- Same as fixed-length bursts, but with address wrapping.
- Useful for cache-line operations or circular buffers.
- Address wraps back to the starting boundary after reaching the burst length.

Note: WRAP4, 32-bit example — starting at address 0x20, the sequence is 0x20, 0x24, 0x28, 0x2C, then wraps back to 0x20.

Simple Transfer Example**Read Transfer****Figure 3-1 Read transfer***Figure 1 — Read transfer*

Write Transfer

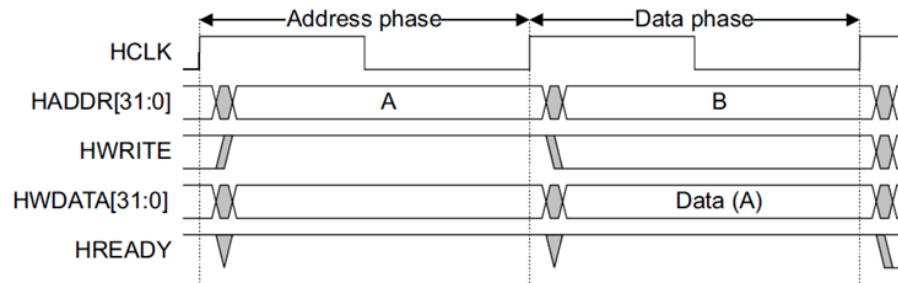


Figure 3-2 Write transfer

Figure 2 — Write transfer

In a simple transfer with no wait states:

- The master drives the address and control signals onto the bus after the rising edge of HCLK.
- The slave then samples the address and control information on the next rising edge of HCLK.
- After the slave has sampled the address and control it can start to drive the appropriate HREADY response.

This response is sampled by the master on the third rising edge of HCLK.

Our Implementation

General Structure

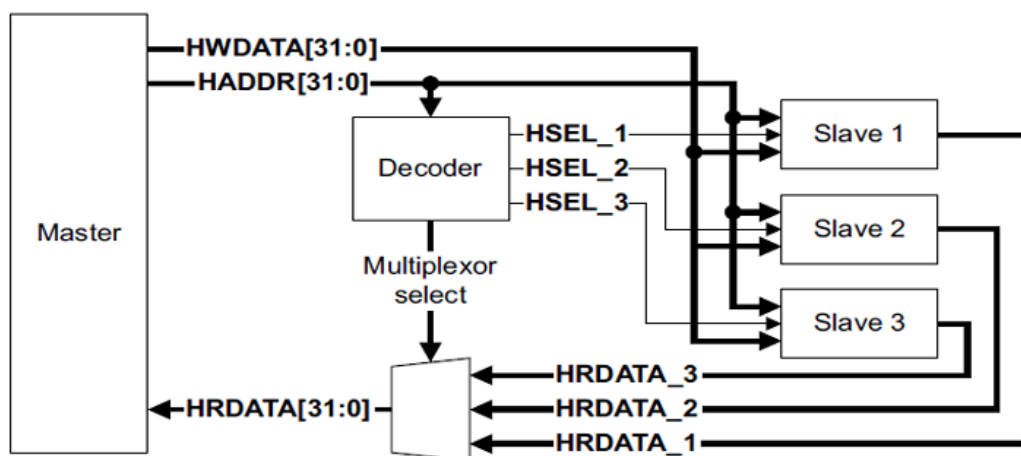


Figure 1-1 AHB-Lite block diagram

Figure 3 — AHB-Lite reference block diagram

Implemented Structure

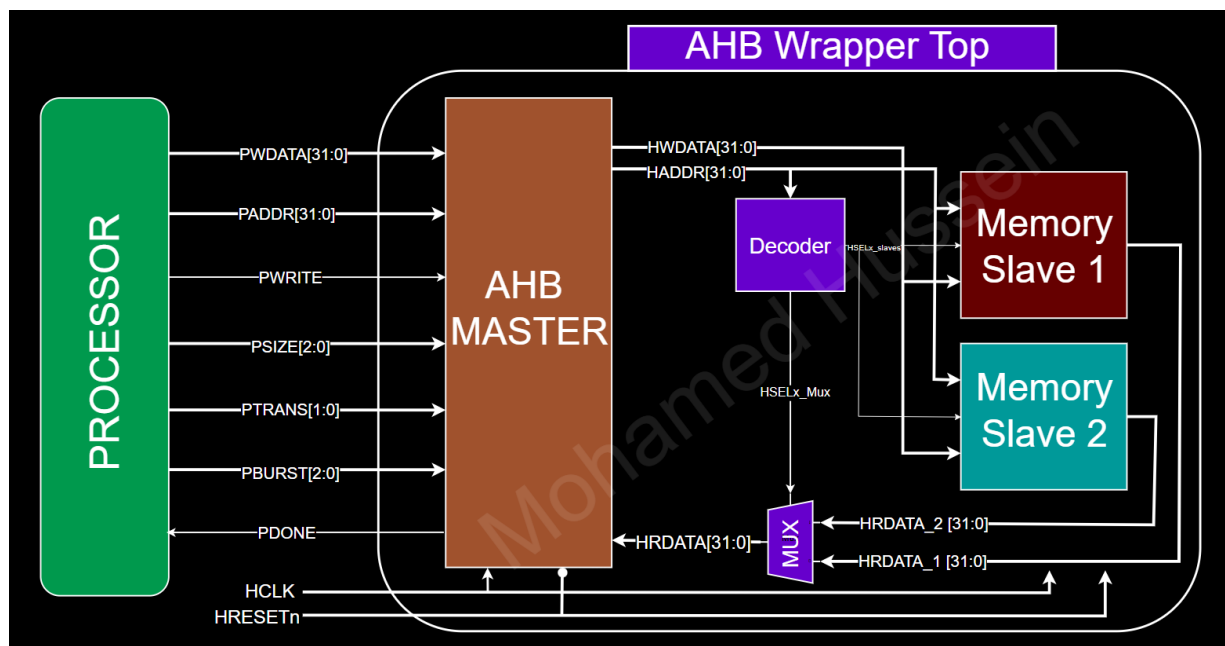


Figure 4 — AHB Wrapper top-level structure

Note: only the essential and most commonly used burst types were implemented in this project.

Supported

- SINGLE transfer (HBURST = 3'b000)
- INCR (incrementing burst) (HBURST = 3'b001)

These two cover:

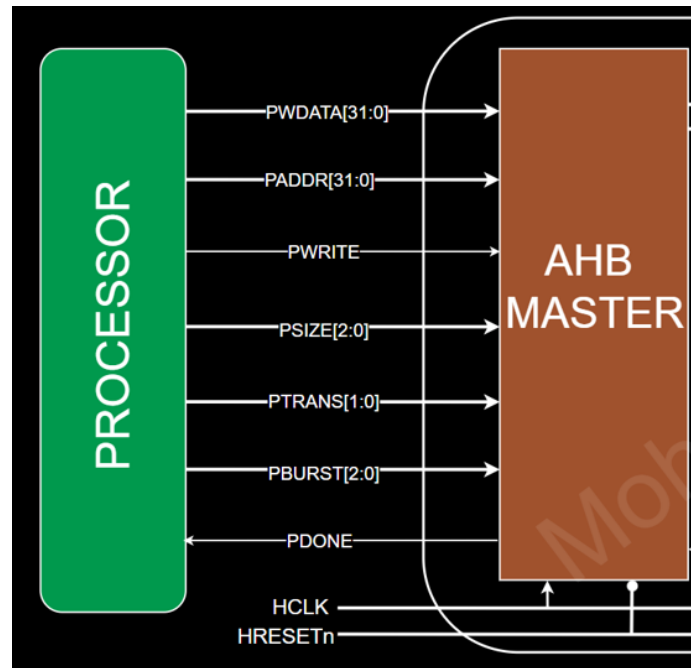
- Arbitrary memory access
- Multi-beat data movement
- Effective verification of address phase and data phase separation

Explanation

Processor Assumption

An external processor module is assumed to communicate with the AHB Master, enabling realistic end-to-end testing across:

- Processor → AHB Master
- AHB Master → AHB Decoder
- AHB Decoder → AHB Slaves



Processor-to-master signal interface

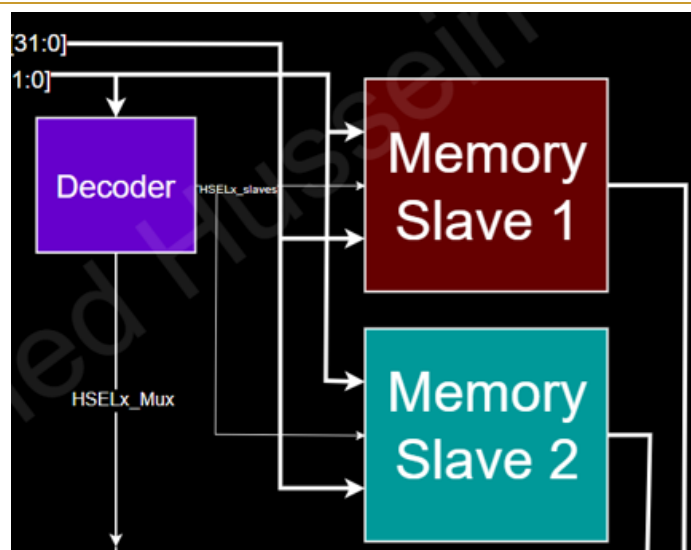
Processor-to-Master Interface

The processor issues transactions to the Master using control signals such as PADDR, PWDATA, PWRITE, PTRANS, PBURST, etc. These signals stimulate the Master, mimicking typical processor behavior.

PDONE Signal

A custom signal PDONE is assumed as an output from the AHB Master back to the processor. It acts as a “transaction complete” flag, going high after every successful AHB transfer, allowing the processor to:

- Synchronize operations
- Avoid issuing overlapping transactions



Dual-slave architecture

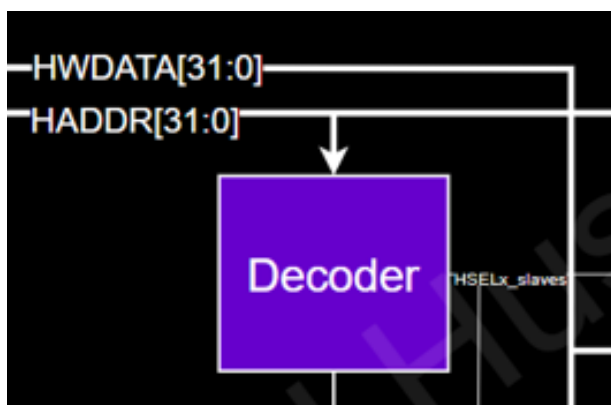
Dual-Slave Architecture

While the original AHB documentation often shows three or more slaves, this implementation includes:

- AHB_Slave_1: a regular RTL-coded slave, acting as a larger memory for general use.
- AHB_Slave_2: a smaller memory block, specifically included to test slave switching and decoder functionality.

Slave 1 — Regular RTL Memory

- Designed in a straightforward RTL style.
- Acts like a general-purpose memory.
- Receives a full range of test scenarios in the testbench (single and burst transfers, various sizes).



Decoder logic

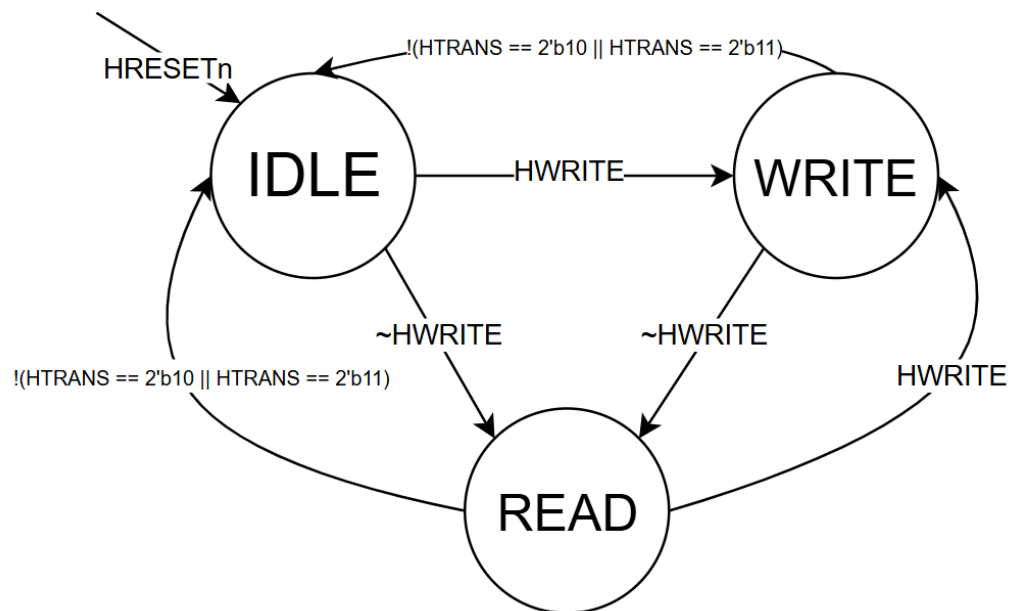
Decoder Logic

A decoder connects the master to slaves and routes transactions using address bits:

- HADDR[31:28] (or appropriate top bits) are used to generate HSELx signals.
- Only one slave is selected at a time, based on the address range.

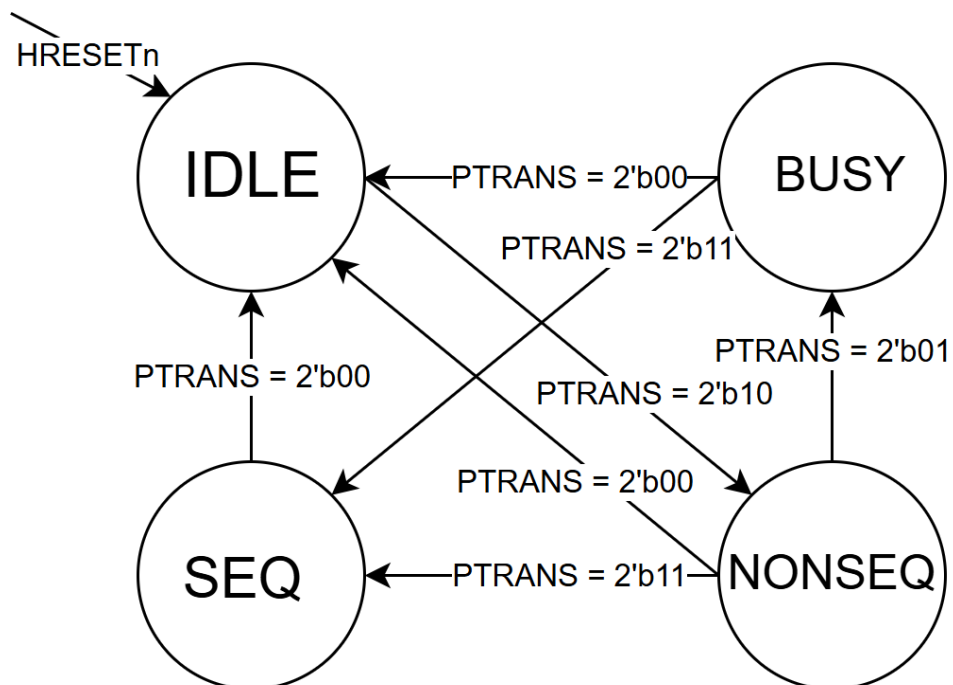
Slave 2 — FSM-Based Implementation

- Contains a smaller memory.
- RTL implemented using a Finite State Machine (FSM) coding style.
- Highlights that the same AHB functionality can be realized using alternative design approaches.
- Receives lighter test stimulus in the testbench due to reduced size.

Slave 2 FSM**Design Modularity**

The structure was kept modular to:

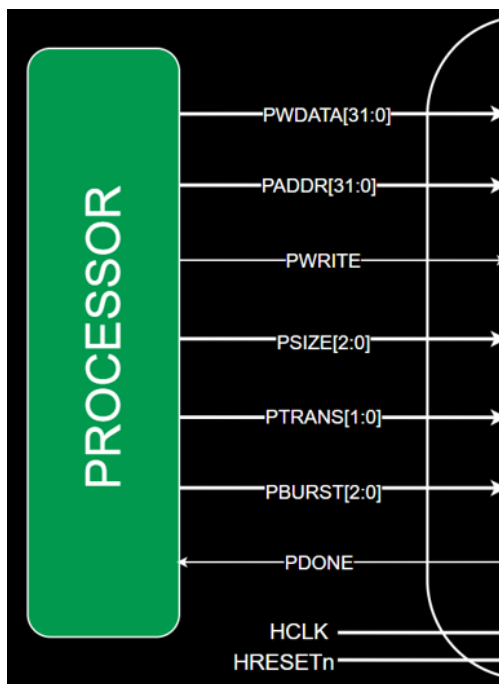
- Simplify testing and debugging.
- Facilitate adding more slaves or peripherals in future iterations.

Master FSM

Verifying Functionality

Flow of the AHB-Lite Testbench

1. Mimicking the Processor



This testbench simulates a processor that interacts with the AHB-Lite bus. Instead of hardcoding test logic inside the master, control signals are exposed such as:

- PADDR: Address to access
- PWDATA: Data to write
- PWRITE: Read or write selection
- PSIZE, PTRANS, PBURST: Transaction type and size

This allows the testbench to act like a processor, sending address and data simultaneously, even though AHB-Lite requires a separation between address and data phases. This mimics realistic system behavior because:

- The processor doesn't know the bus protocol; it simply issues transactions.
- The master handles protocol details, breaking the transaction into address and data phases.

This setup allows simulation of end-to-end system behavior across master, decoder, mux, and slaves.

Note: although the testbench sends both PADDR and PWDATA in the same cycle (cycle N), the master module ensures HADDR appears at cycle N and HWDATA at cycle N+1, respecting the address phase → data phase separation required by the AHB-Lite protocol.

2. Waveform Comparison: Expected vs. Implementation

Standard AMBA AHB-Lite protocol waveforms from documentation are compared with simulation results, across two categories:

a. Single Transfers (HBURST = 3'b000)

Expected Write Transfer

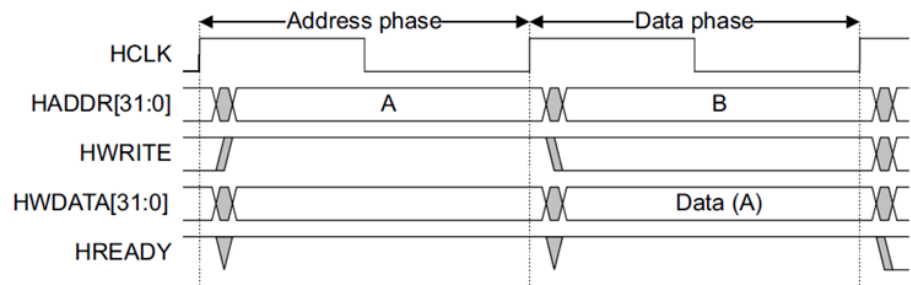
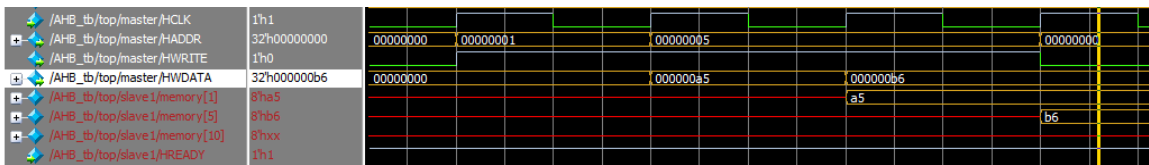


Figure 3-2 Write transfer

Figure 5 — Write transfer (reference)

Waveform Generated



Expected Read Transfer

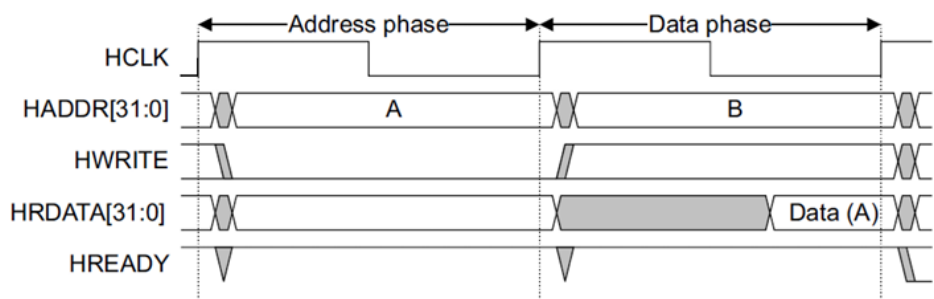
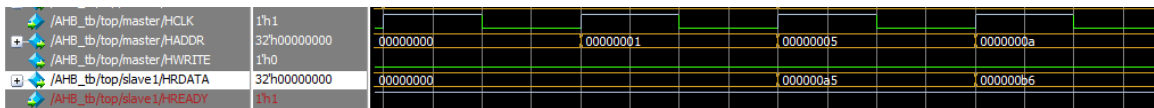


Figure 3-1 Read transfer

Figure 6 — Read transfer (reference)

Waveform Generated



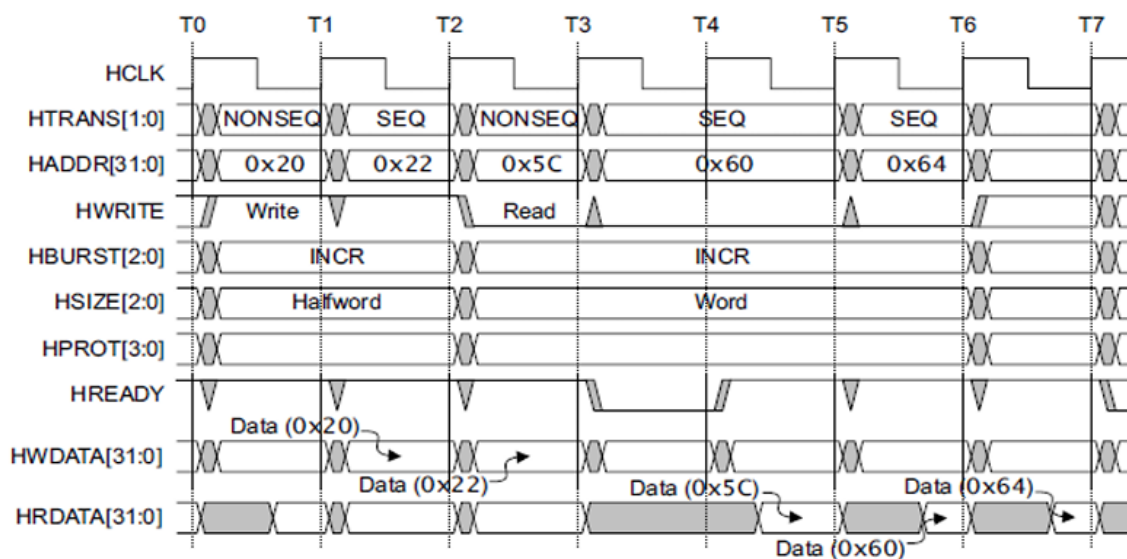
From these two comparisons:

- Write transfer: the generated waveform matches the expected waveform in the AHB documentation.
- Write transfer: HADDR appears at cycle N, HWDATA at cycle N+1, and data is accessed by the slave at cycle N+2 — exactly as the specification states.
- Read transfer: the generated waveform matches the expected waveform in the AHB documentation.
- Read transfer: HADDR appears at cycle N and HRDATA at cycle N+1 — exactly as the specification states.

b. Incrementing Burst Transfers (HBURST = 3'b001)

- HTRANS sequence: first NONSEQ, then several SEQ.
- Master auto-increments the address by size: +1 for 8-bit, +2 for 16-bit, +4 for 32-bit.

Expected Write & Read Transfers

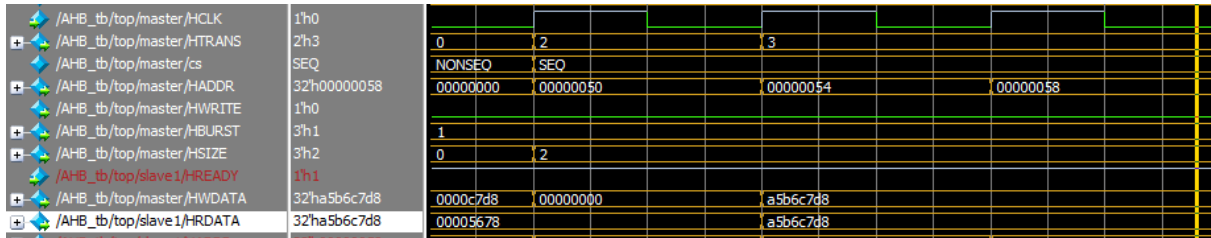


Waveform Generated (Write)



- Write transfer: the generated waveform matches the expected waveform in the AHB documentation.
- HADDR appears at cycle N, HWDATA at cycle N+1, and data is accessed by the slave at cycle N+2 — exactly as the specification states.
- With HSIZE = 2'b01 (16-bit transfer), the address increments by 2 every cycle.
- Memory access shows the 16-bit write completing successfully at cycle N+3.

Waveform Generated (Read)



- Read transfer: the generated waveform matches the expected waveform in the AHB documentation.
- HADDR appears at cycle N and HRDATA at cycle N+1 — exactly as the specification states.
- With HSIZE = 2'b10 (32-bit transfer), the address increments by 4 every cycle.
- The read data does not change between the two addresses because the same data was written to both.

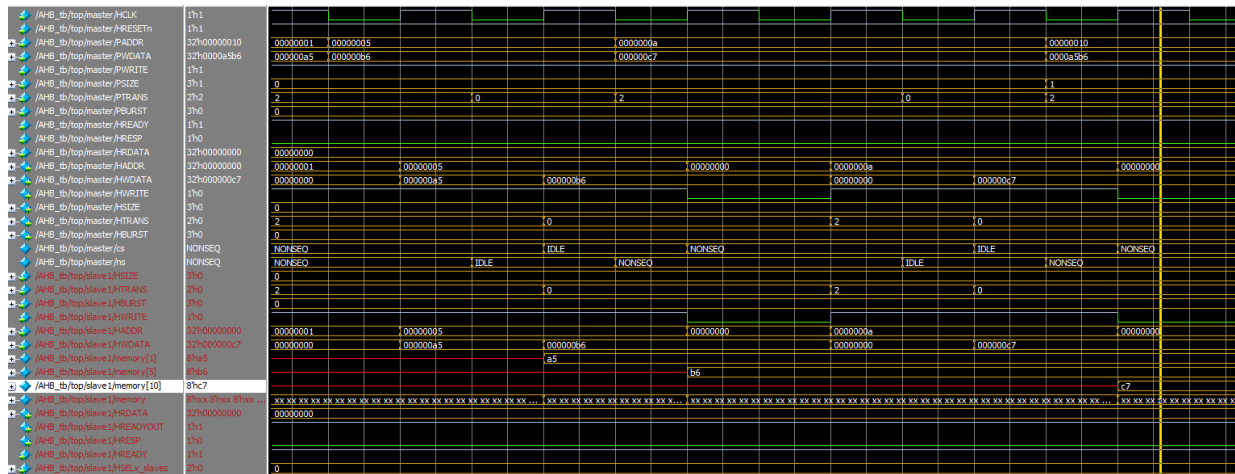
3. Verification of Slave 1

All of the following scenarios are validated using Slave 1:

- Writing to several locations using a single transfer (HBURST = 0).
- Reading back from the exact same addresses using a single transfer.

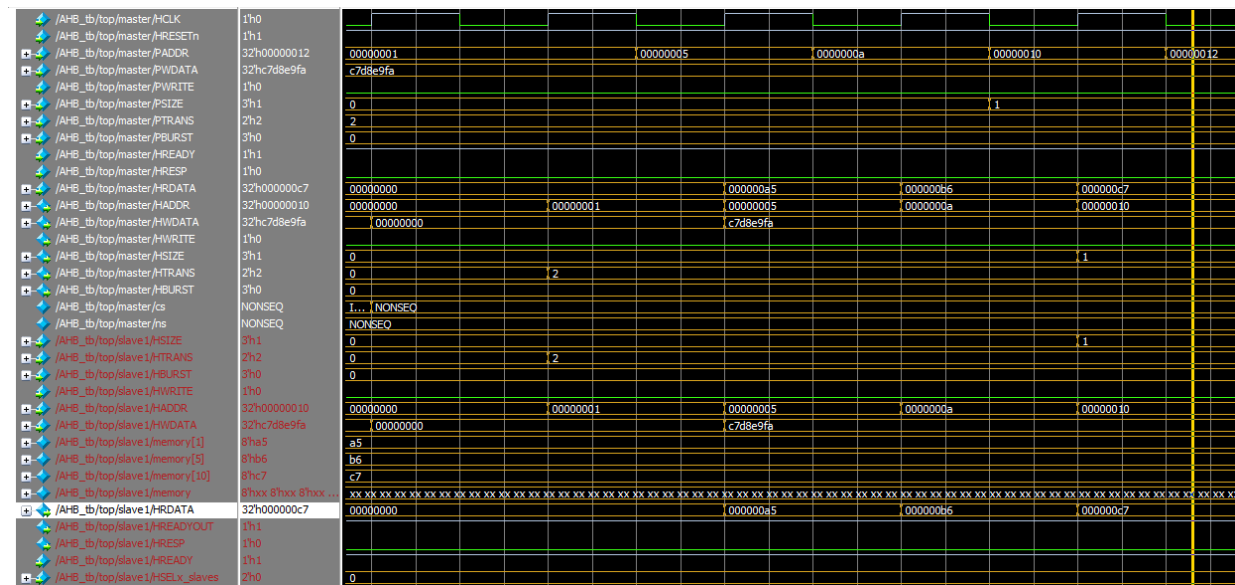
Single Transfers — 8-bit (addresses 1, 5, 10)

Writing:



Data was written to three different addresses, respecting the address phase and data phase.

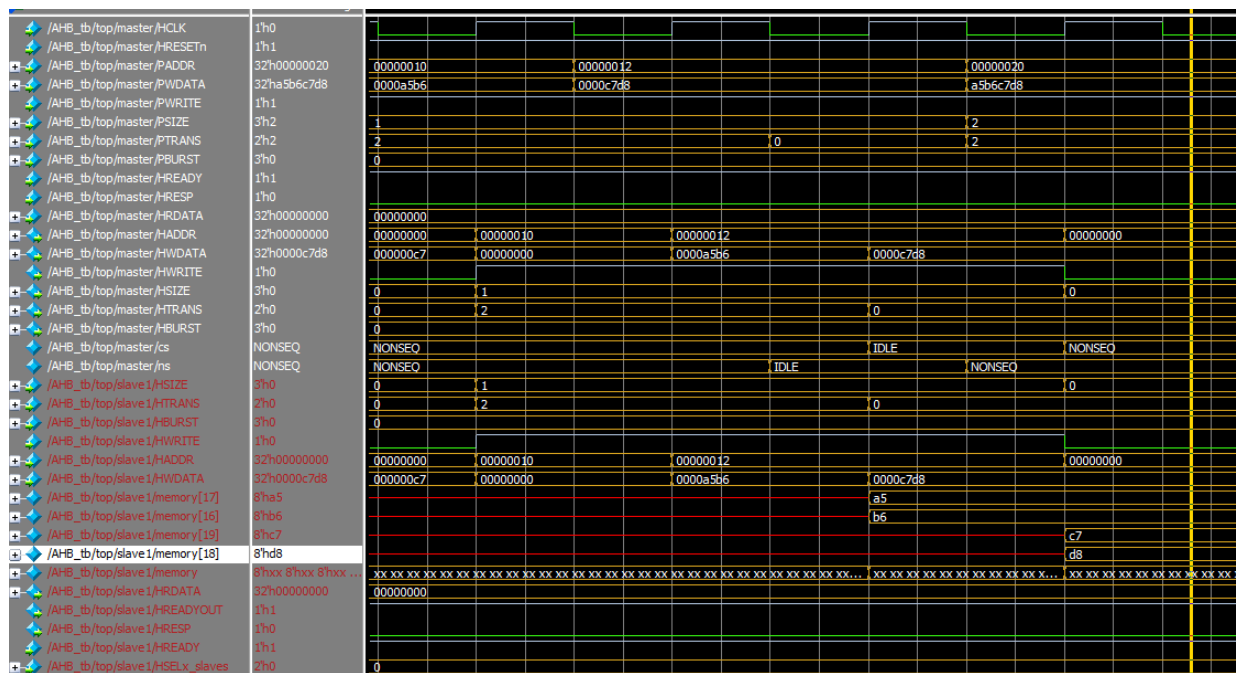
Reading:



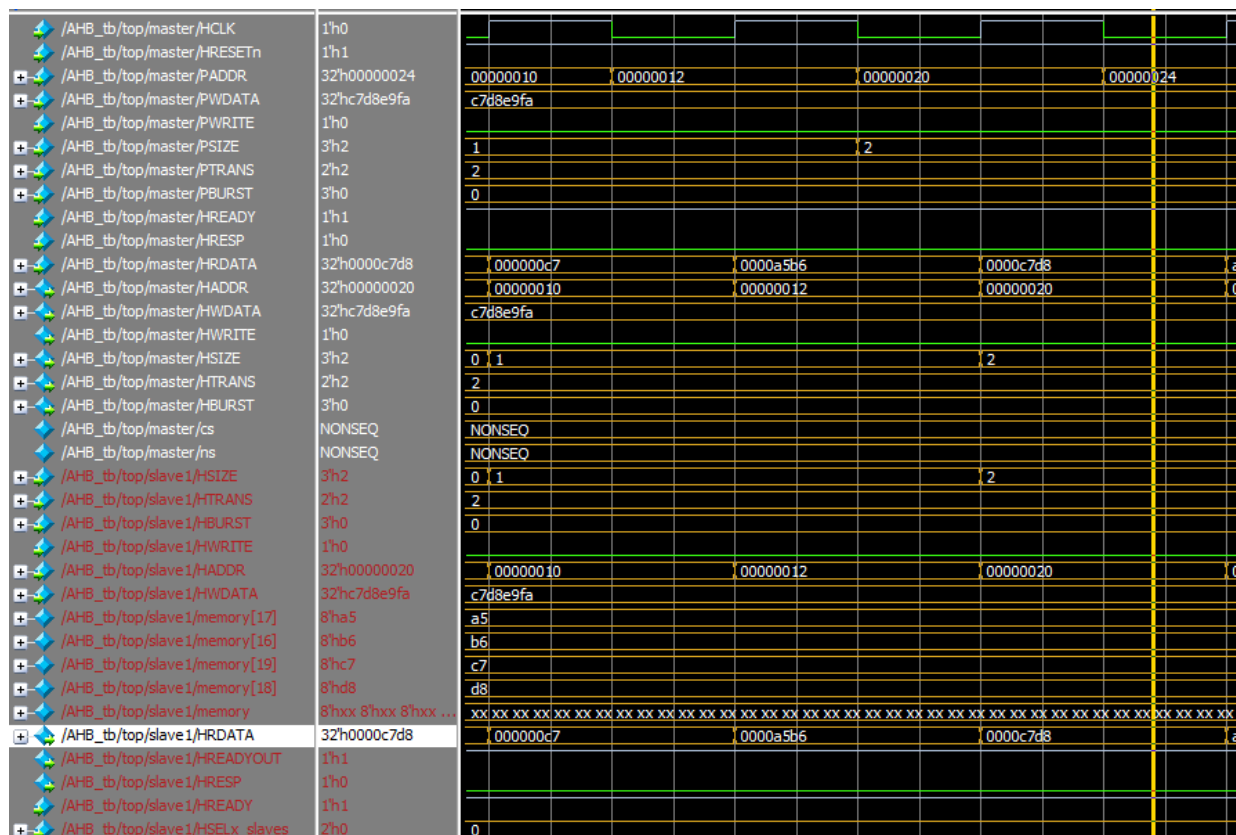
HRDATA shows the same values written before, again respecting the address phase and data phase (see HADDR and HRDATA).

Single Transfers — 16-bit (addresses 16, 18)

Writing:



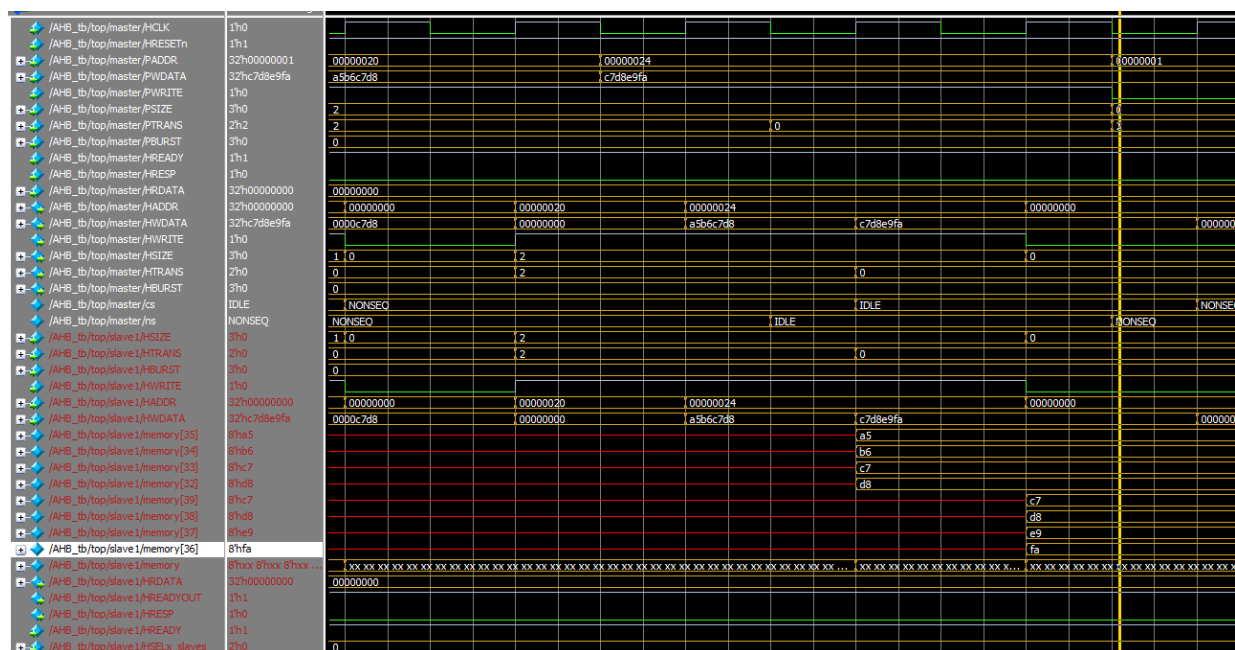
Reading:



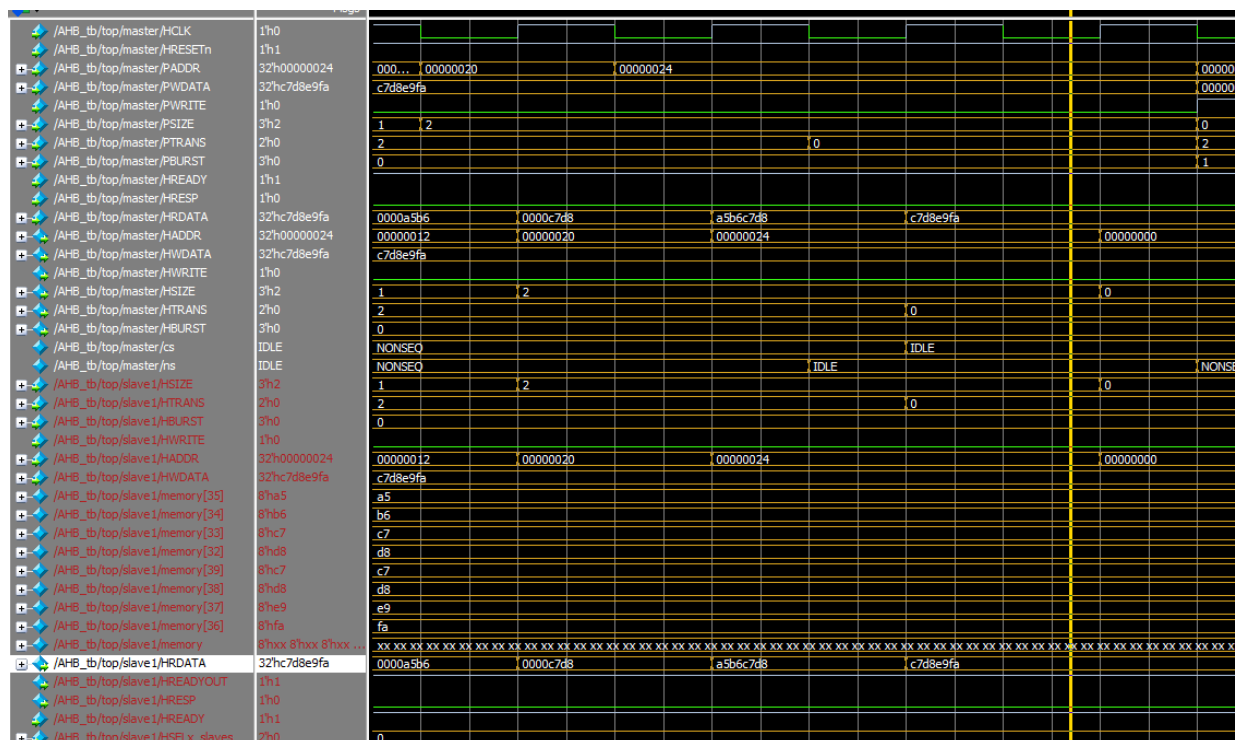
HRDATA matches the previously written values, with address phase and data phase correctly respected.

Single Transfers — 32-bit (addresses 32, 36)

Writing:



Reading:

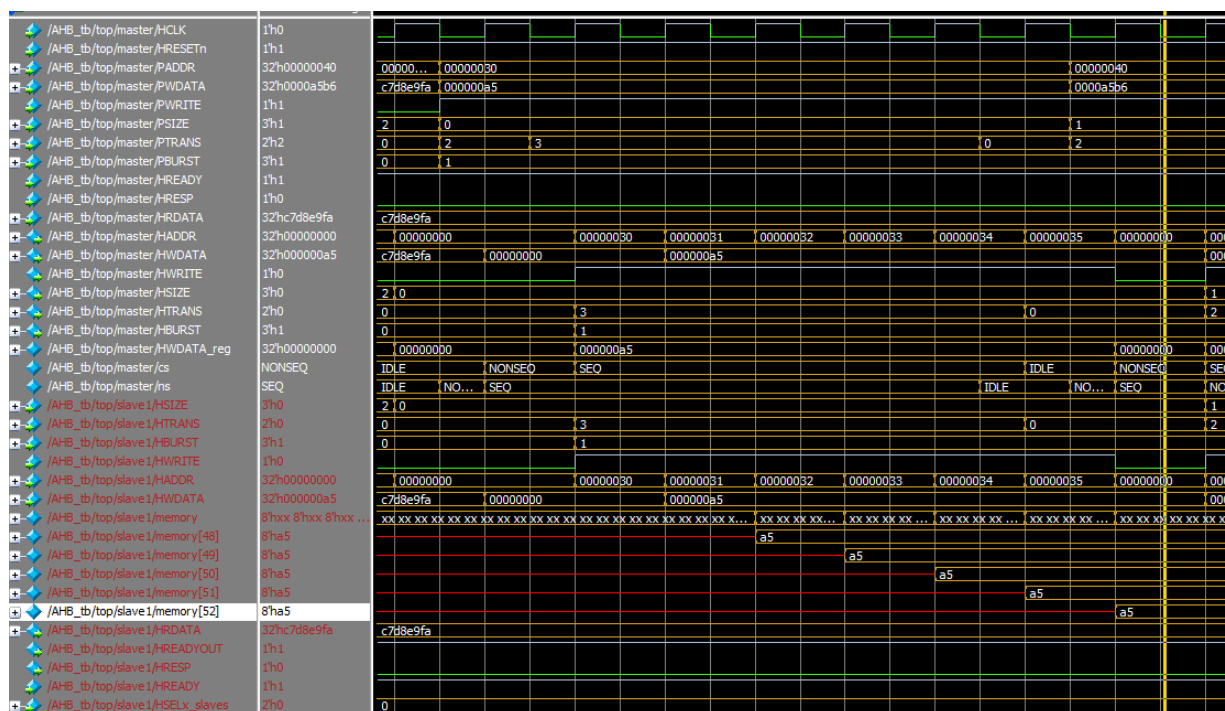


HRDATA again matches the values written previously.

- Data is read back and matches written data, confirming correct storage.
- The HSELx_slaves signal remains 2'b00 throughout the verification of Slave 1.

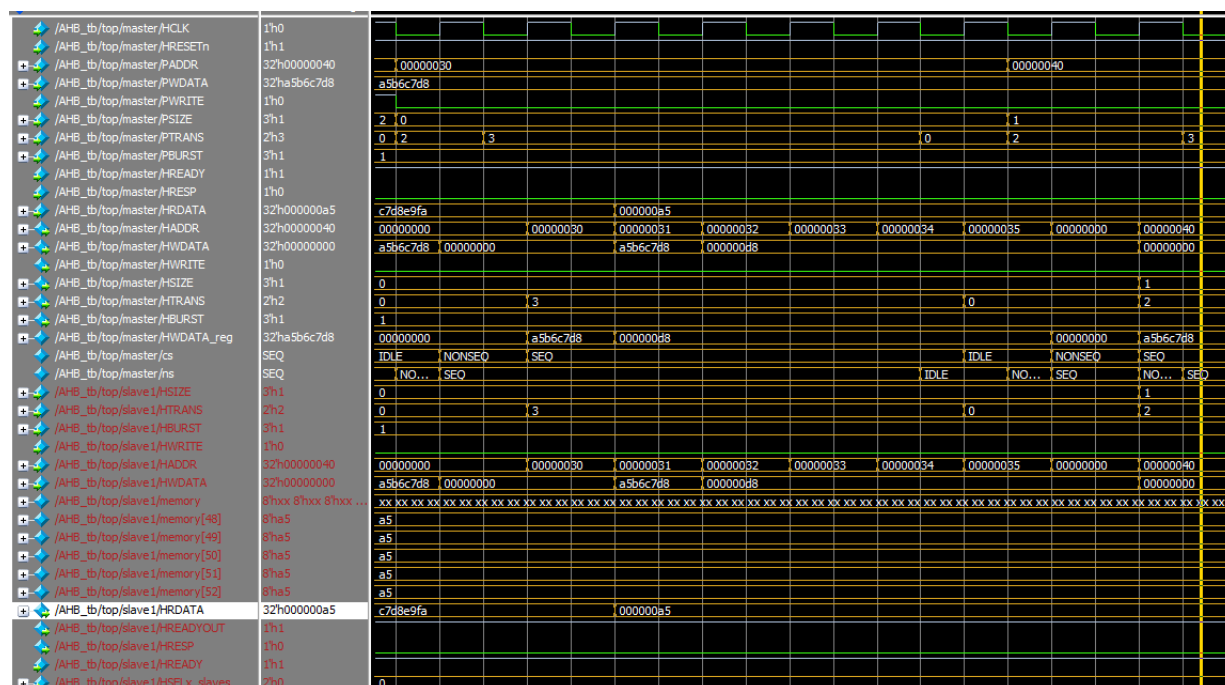
Incrementing Burst Transfers — 8-bit (5 sequential bytes, start address 48)

Writing:



- Data was written across the addressed range, respecting address phase and data phase.
- The address increments automatically as expected for an incrementing burst — the start address 0xa5 was written across five sequential addresses.

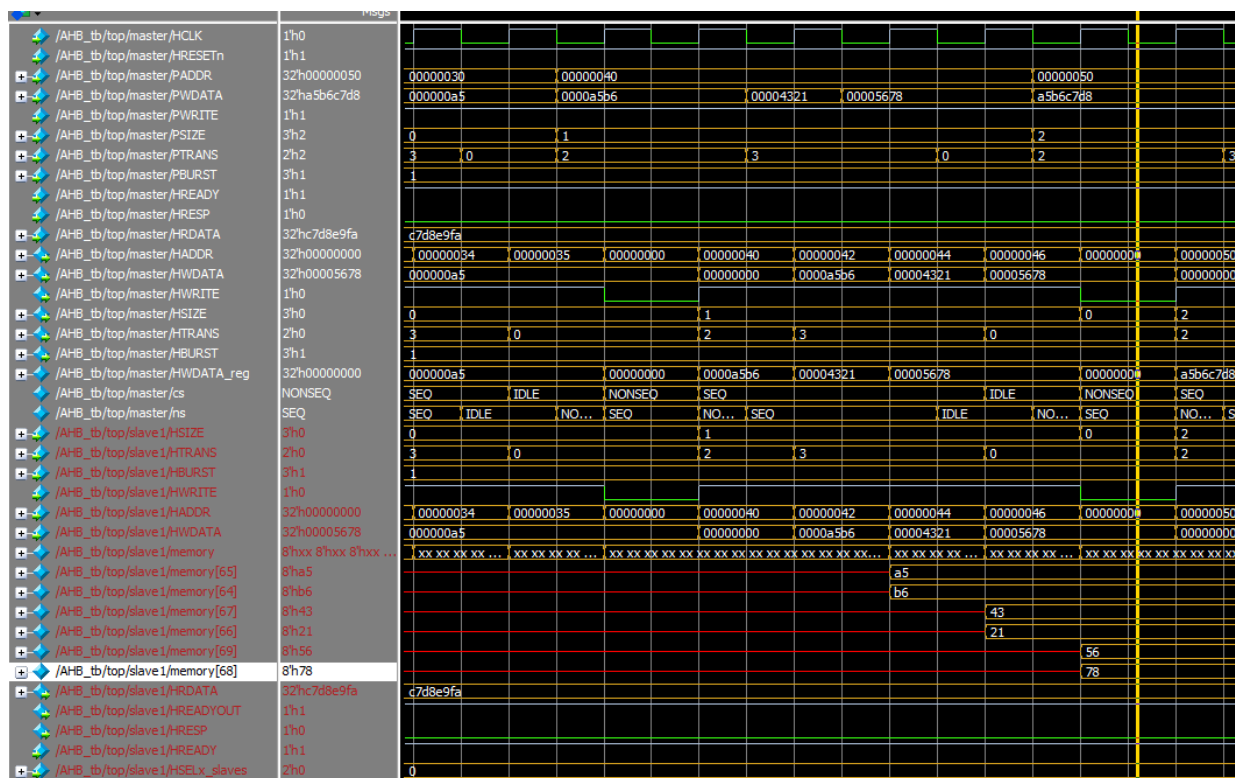
Reading:



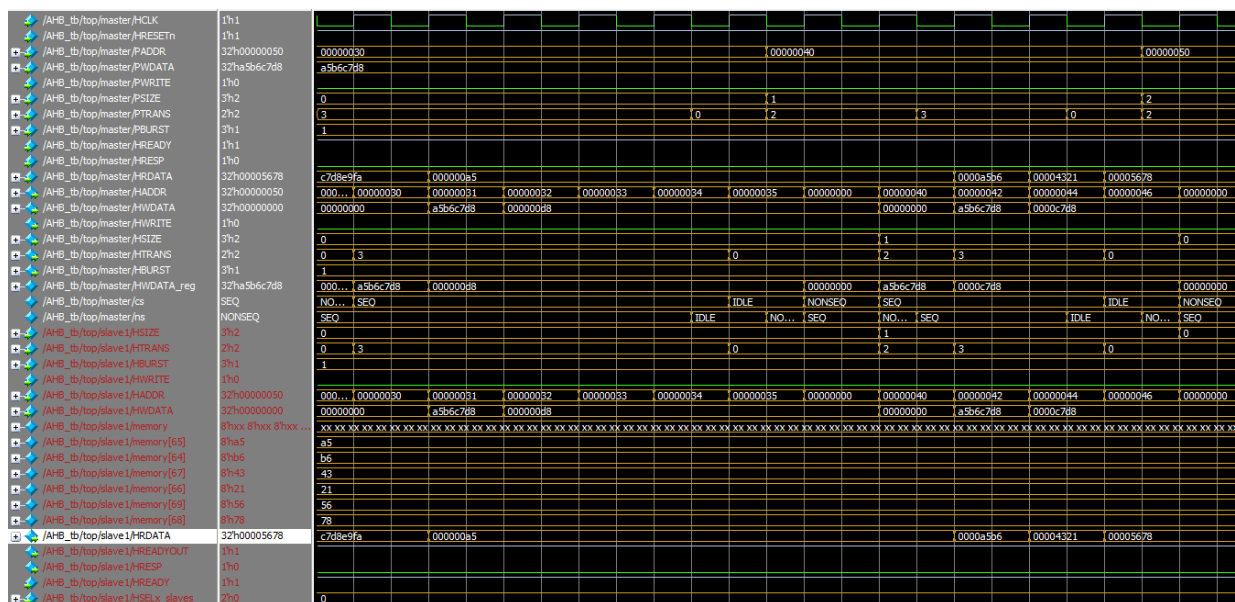
HRDATA matches the value written before (0xa5), again respecting address phase and data phase.

Incrementing Burst Transfers — 16-bit (3 sequential halfwords, start address 64)

Writing:



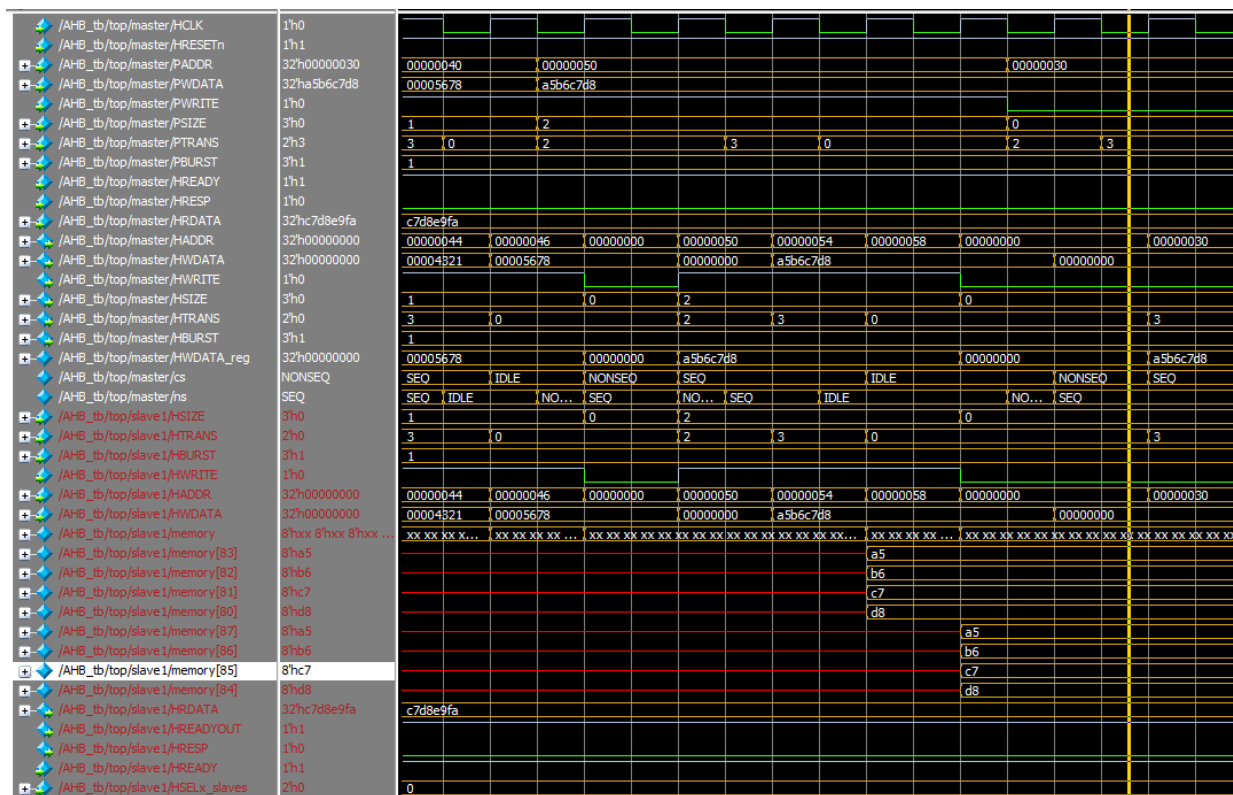
Reading:



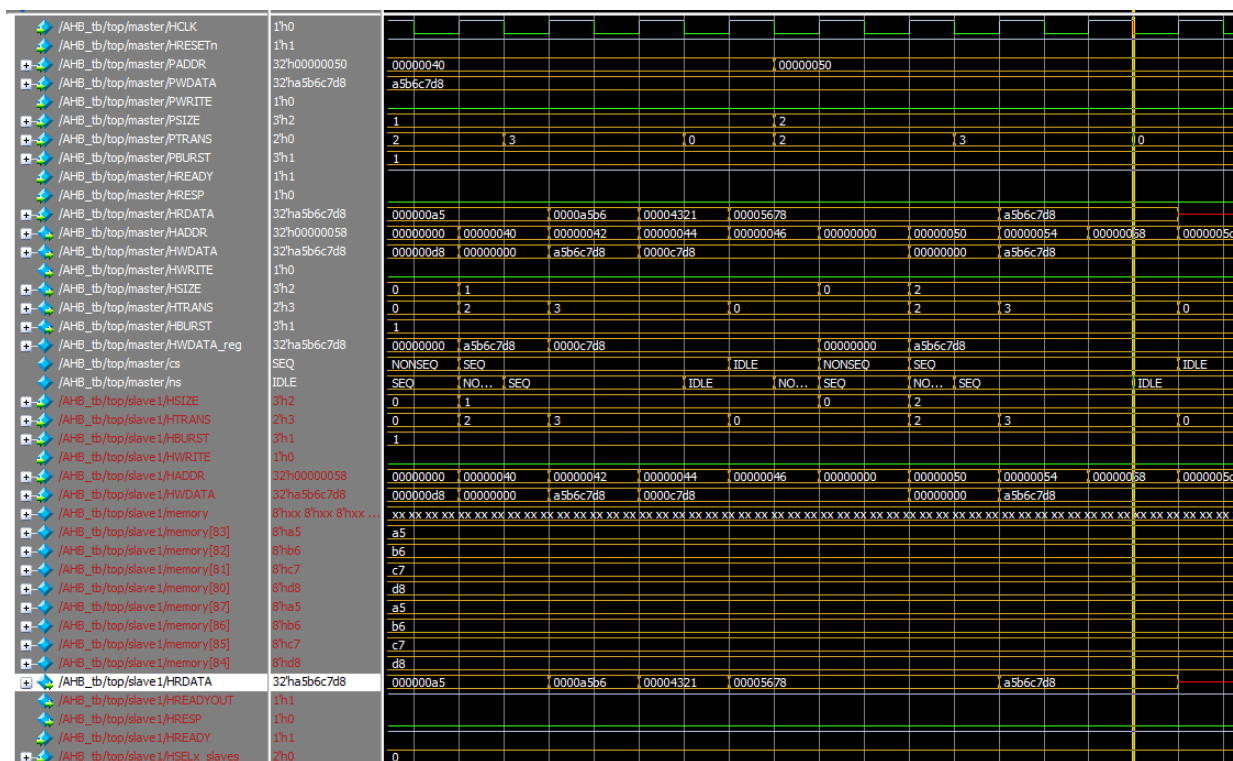
HRDATA matches the previously written values (different values across the three addresses).

Incrementing Burst Transfers — 32-bit (2 sequential words, start address 80)

Writing:



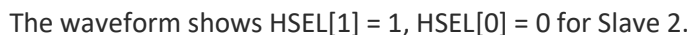
Reading:



HRDATA matches the previously written value (0xa5b6c7d8).

- Data is read back and matches written data, confirming correct storage.
- The HSELx_slaves signal remains 2'b00 throughout the verification of Slave 1.
- In incrementing bursts, the address increments based on the HSIZE value.
- All transactions are verified via waveform observation, and HRDATA matches the expected output.

Changing the address range to select Slave 2 (e.g., 32'h4000_0000), the decoder logic activates the correct slave using HSEL.

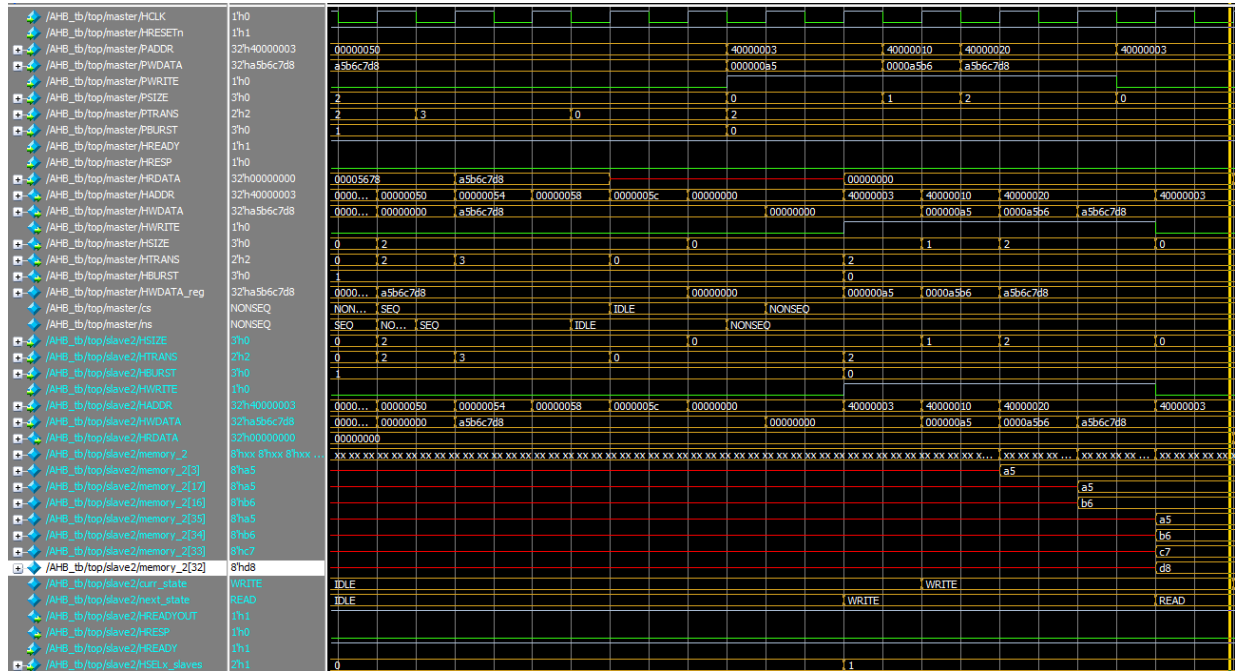


- Smooth switching between slaves indicates the architecture can scale to additional slaves.

5. Verifying Slave 2

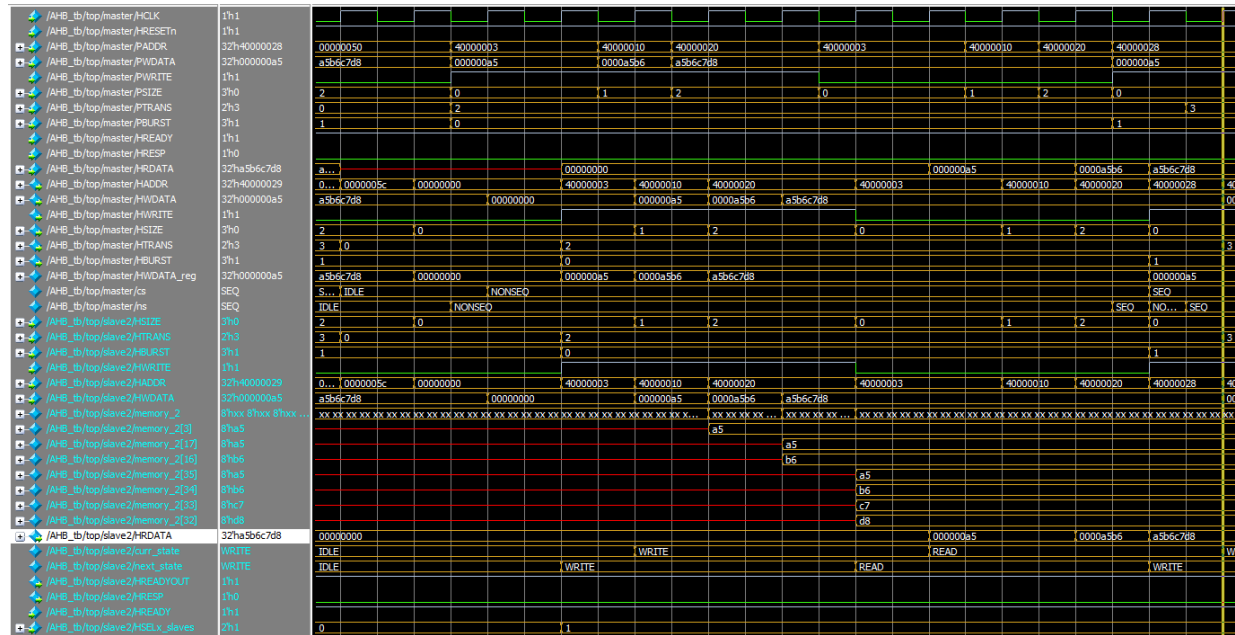
The smaller memory size results in lighter testing: 8-, 16-, and 32-bit single transfer write/read.

Writing:



Data was written to three different addresses, respecting address phase and data phase.

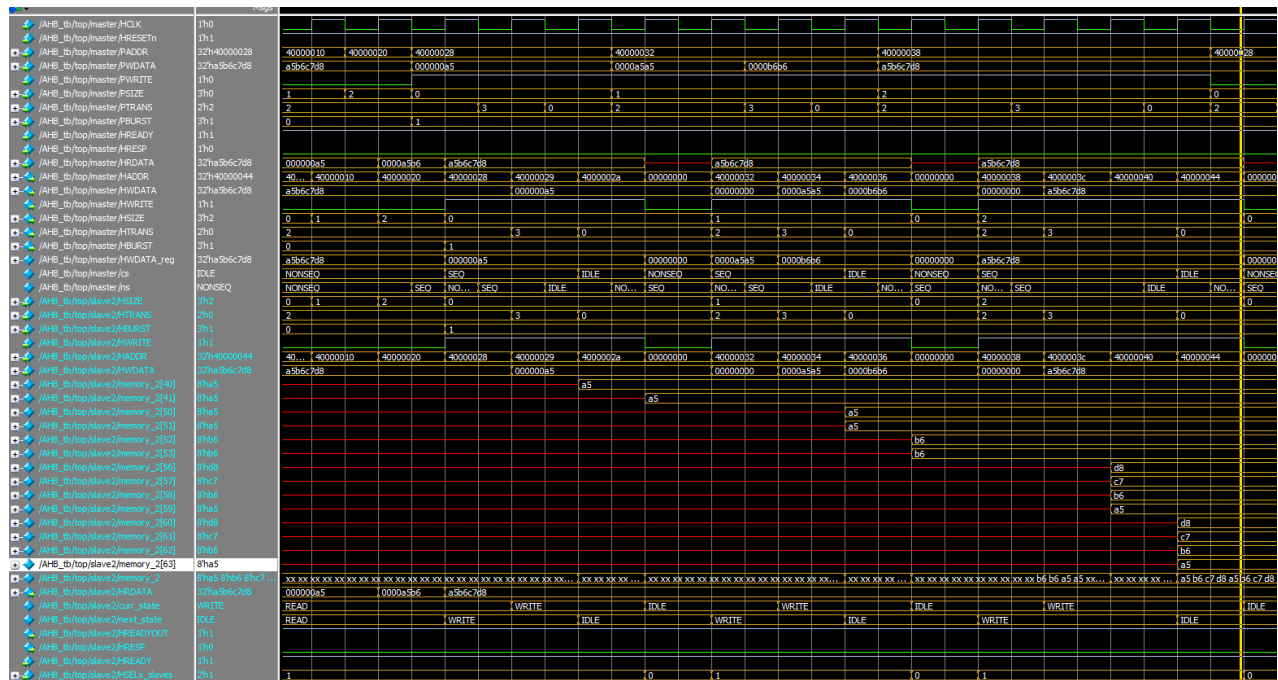
Reading:



HRDATA matches the values written before, respecting address phase and data phase.

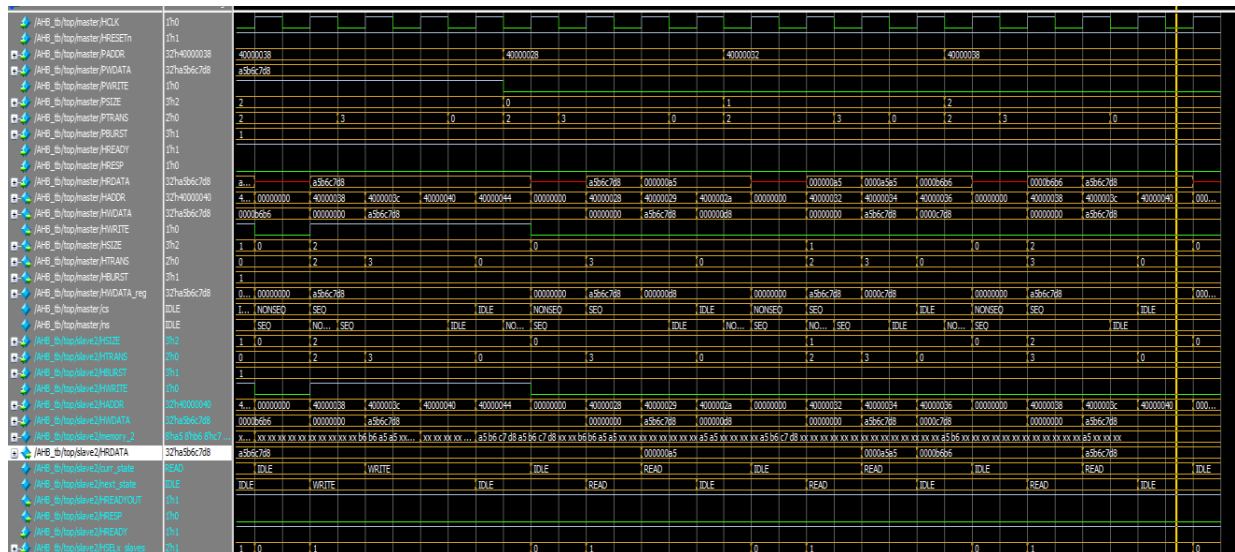
8-, 16-, and 32-bit Incrementing Burst Write/Read

Writing:



- Data was written to three different addresses, respecting address phase and data phase.
- The address increments automatically based on the HSIZE value: if 0, the address increments by 1; if 1, by 2.

Reading:



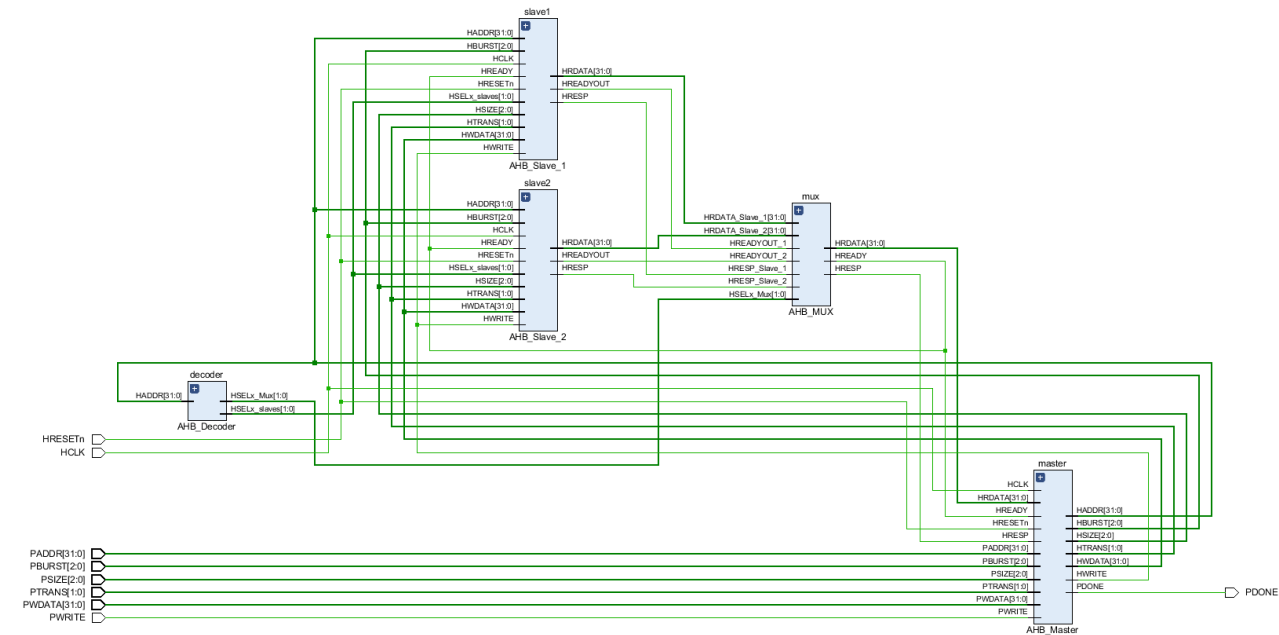
HRDATA matches the values written before.

- Data is read back and matches written data, confirming correct storage.
- The HSELx_slaves signal is 2'b01 throughout the verification of Slave 2 (except in the IDLE state).
- In incrementing bursts, the address increments based on the HSIZE value.

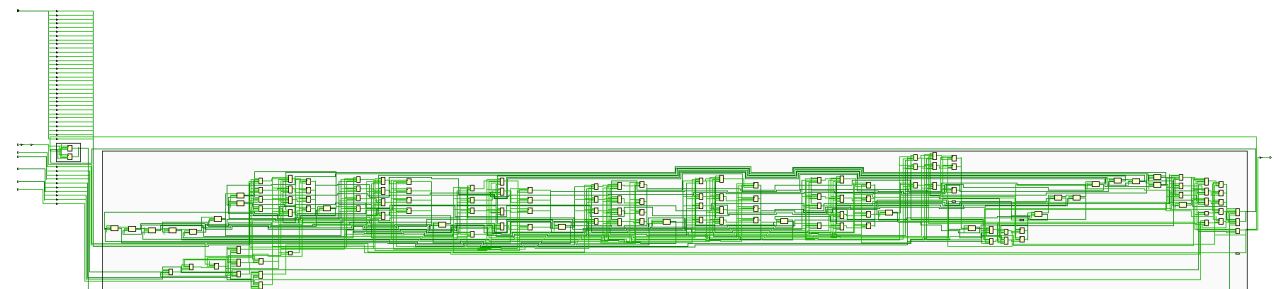
[illegible]

FPGA Flow Using Vivado

Elaboration



Synthesis



References

- AMBA 3 AHB-Lite Protocol Specification, University of Michigan.
- Simon Southwell, “SoC Bus and Interconnect Protocols #1: Busses (APB and AHB),” LinkedIn.